

Diseño Avanzado de Algoritmos
Actividad 1 – Diseño, análisis y comparación de algoritmos
Bácher en Informática – UNIPRO
Alumno: Eduardo ROBLED0 MARTÍNEZ
Repositorio GitHub: <https://github.com/erobmar/AnalizadorLCS>
Profesor: Bernardo RONQUILLO JAPÓN
Fecha: 08/12/2025

Contenido

1.- Introducción	2
2.- Metodología	2
2.1.- Entorno de desarrollo y ejecución	2
2.2.- Implementación de los algoritmos.....	3
2.3.- Generación del conjunto de datos	3
2.4.- Ejecución de pruebas	4
3.- Resultados	4
3.1.- Comportamiento para tamaños pequeños (menos de 20)	4
3.2.- Comportamiento para tamaños grandes (20 o más)	5
4.- Discusión de los resultados	6
5.- Conclusiones.....	7
6.- Anexos	8
6.1.- Anexo 1 – Tabla de resultados.....	8
6.2.- Enlace a repositorio.....	9

1.- Introducción

La presente actividad aborda el problema de la Subsecuencia Común más Larga (en adelante LCS, del inglés Longest Common Subsequence). Este es un problema habitual en el ámbito académico para afianzar conocimientos sobre diseño de algoritmos y programación dinámica y consiste en, dadas dos cadenas de caracteres, encontrar la subsecuencia común entre ellas más larga. No es necesario que los elementos de esta subsecuencia sean contiguos (dado que entonces estaríamos hablando de subcadenas en lugar de subsecuencias) pero sí que deben mantener el mismo orden.

Sus aplicaciones son numerosas e importantes. Desde la comparación de secuencias de ADN hasta el Big Data - no directamente como se implementará en esta actividad pues sería demasiado costoso sino con algunas versiones heurísticas más apropiadas para el cómputo a gran escala.

Las versiones del algoritmo que se analizarán en este documento son:

- **Recursividad directa sin memorización:** Solución sencilla pero que, computacionalmente, escala con rapidez.
- **Divide y vencerás de Hirschberg:** Más compleja en su implementación pero con una mejora sustancial en eficiencia temporal, lo que permite trabajar con conjuntos de datos mucho mayores. A cambio, el consumo de memoria es mucho mayor.
- **Programación dinámica tabular:** Mantiene la complejidad temporal teórica de Hirschberg ($O(n \cdot m)$) pero su implementación más directa proporciona unos tiempos de ejecución ligeramente mejores. Como contrapartida, su consumo de memoria es notablemente mayor.

El objetivo de esta actividad es, precisamente, comparar el rendimiento de estos tres algoritmos LCS y, con datos empíricos llegar a una conclusión sobre cuál es el más idóneo.

Se han implementado las tres versiones mencionadas en Python 3 sin usar librerías específicas para el tratamiento de este problema. Posteriormente se ha generado un dataset de prueba con cadenas de distintos tamaños, que son procesadas por cada algoritmo mientras se mide el uso de CPU y memoria para determinar su eficiencia.

El código fuente de esta actividad se puede descargar y probar en el repositorio de GitHub del autor (<https://github.com/erobmar/AnalizadorLCS>). La ejecución es tan sencilla como, una vez descargados los archivos 'dataset.csv' y LCS.py, teclear en línea de comandos 'python3 LCS.py' en entornos Linux con dicha versión instalada o 'python LCS.py' en entornos Windows.

2.- Metodología

2.1.- Entorno de desarrollo y ejecución

Tanto el desarrollo como las pruebas se han llevado a cabo en un ordenador portátil con las siguientes características:

- Procesador: AMD Ryzen 9 8945HS
- Memoria RAM: 32GB
- Sistema Operativo: Windows 11
- Lenguaje e IDE: Python 3.10.6 sobre Visual Studio Code 1.106.3

El uso de librerías se ha limitado a la biblioteca estándar de Python para mantener la portabilidad.

2.2.- Implementación de los algoritmos

- **LCSRecursoivo:** El algoritmo más básico; recursivo sin memorización. Se explicará brevemente a continuación:
 - Caso base: Si alguna de las dos cadenas tiene longitud cero, devuelve una cadena vacía porque, evidentemente, ya no se pueden añadir caracteres a la LCS.
 - Caso recursivo: Se presentan dos casos:
 - Si $A[i-1]$ es igual a $B[j-1]$, implica que el carácter forma parte de la LCS y se añade a la siguiente llamada recursiva, sobre los caracteres $(i-1)$ y $(j-1)$.
 - Si no lo son, se prueban dos opciones; eliminando el último carácter de A y eliminando el último carácter de B. Se comparan ambos resultados y se devuelve la opción más larga.
- **LCSDivide:** Aplica el algoritmo de “divide y vencerás” de Hirschberg:
 - Se divide la cadena A en dos mitades A1 y A2
 - Se calcula el vector de longitudes de LCS hacia delante de A1 con todos los prefijos de B usando la función `divideAdelante`.
 - Se hace lo mismo pero hacia atrás con A2 sobre B usando la función `divideAtras`.
 - Se busca qué índice de esos dos vectores maximiza su suma. Ese será k; el punto de corte para la cadena B que garantiza la LCS se puede reconstruir uniendo las LCS de (A1, B1) con la de (A2, B2).
 - Se llama recursivamente a LCSDivide sobre (A1, B1) y sobre (A2, B2) y se concatenan los resultados.
- **LCS Dinámico:** Implementa programación dinámica tabular.
 - Crea una matriz de tamaño $[n+1] \times [m+1]$ dado que, además de los caracteres de las cadenas, debe contabilizar el caso de “” (cadena vacía).
 - En esa matriz se van almacenando las longitudes de la LCS entre los prefijos de A y B. Se rellena iterativamente propagando el máximo entre cada celda superior e izquierda.
 - Una vez rellena la matriz, se toma el elemento de la última posición (abajo a la derecha) que, por definición, contendrá la longitud máxima de la LCS. A partir de este elemento se reconstruirá la LCS de la siguiente manera:
 - Se comprueba si los caracteres de los índices de ese elemento son iguales.
 - Si lo son, se añade el elemento a la LCS y se comprueba en diagonal hacia arriba y a la izquierda.
 - Si no lo son, se comparan los valores de las “casillas” contiguas a esta, a la izquierda y encima, y se comprueba la de mayor valor.

Los tres algoritmos se han diseñado, además, para que no solo calculen la longitud de la LCS sino que, además, devuelvan su valor. De este modo se puede comprobar que los resultados son coherentes entre algoritmos.

2.3.- Generación del conjunto de datos

Se explica de forma muy breve por no ser parte de la materia en sí. Para realizar las pruebas se precisó de un dataset creado por una función en Python que genera caracteres al azar dentro de un conjunto de caracteres dado – para esta actividad, el conjunto de las letras mayúsculas – y que se ejecutó dos veces para cada tamaño de secuencia establecido en el array `tamanyosCadena = [8,9,10,11,12,13,14,15,16,17,20,40,50,100,200,1000,2000]`.

Incluyendo este archivo (dataset.csv) en el repositorio aseguramos la replicación de pruebas por parte de cualquiera que ejecute el programa obteniendo, eso sí, resultados ligeramente diferentes en función de las especificaciones del sistema en que se ejecute.

El programa para la generación del dataset también se incluye en el repositorio a efectos de investigación, aunque no forme parte de la actividad en sí.

2.4.- Ejecución de pruebas

El programa, tras dar la bienvenida al usuario, ofrece tres opciones:

- 1) **Ejecutar pruebas:** Ejecutará cada algoritmo sobre cada una de las entradas del conjunto de datos y almacenará los resultados en un array en memoria. Es requisito imprescindible que el archivo 'dataset.csv' se encuentre en el mismo directorio que LCS.py para poder ejecutar las pruebas. Cabe destacar que, en la función ejecutarPruebas, dentro del case '0', en el primer if, se puede ajustar la profundidad del algoritmo recursivo. Se ha establecido el valor máximo a 17 pues con valores mayores se volvía inasumible computacionalmente. No obstante, se puede experimentar con este valor libremente – siempre teniendo en cuenta sus riesgos. A partir de este valor, el algoritmo se salta dicha prueba y almacena 0 como valor de tiempo de CPU y memoria para esa prueba y las siguientes del algoritmo recursivo. Con dicho valor en 17, las pruebas tardan en completarse unos 4 minutos – en el equipo que se ha desarrollado, el resultado puede variar en función del equipo en que se realicen.

Gracias a las librerías time y tracemalloc, tomamos medidas tanto del tiempo de CPU como de la memoria antes y después de ejecutar cada algoritmo para conocer los recursos consumidos por cada uno. Si bien es cierto que Python no es el lenguaje más preciso para estas mediciones, sí que nos dará una medida aproximada suficiente para poder comparar los algoritmos.

- 2) **Mostrar resultados en pantalla:** Toma el array de resultados almacenado en memoria y lo muestra por pantalla. Obviamente, si las pruebas no se han ejecutado, informará de ello al usuario.
- 3) **Exportar resultados a un archivo csv:** Toma el array de resultados y lo escribe en el archivo 'resultados.csv' informando eventualmente si no se hubieran ejecutado las pruebas con anterioridad. Para el formato de los datos se ha optado por sustituir el separador decimal '.' por ',' para evitar problemas al importar los datos en MS Excel. Como separador entre valores no se usó, obviamente, la coma sino el punto y coma.

3.- Resultados

La tabla completa con los resultados de la ejecución puede verse en el Anexo 1 así como, en bruto, en el archivo 'resultados.csv' del repositorio. A continuación se analizará el comportamiento de los algoritmos en casos representativos y las tendencias observadas.

3.1.- Comportamiento para tamaños pequeños (menos de 20)

En estos casos, para los tamaños más pequeños (8, 9 y 10), los tres algoritmos son totalmente válidos, manteniendo los valores de eficiencia temporal y uso de memoria en valores muy bajos:

- **Recursividad:** En el orden de los milisegundos y con un uso de memoria en torno a los 3KB
- **Hirschberg:** En el orden de los microsegundos y un uso de memoria medio alrededor de los 6KB

- **Dinámico tabular:** También en el orden de los microsegundos pero alrededor del doble de rápido que Hirschberg. Su uso de memoria medio es bastante similar al del anterior pero siempre ligeramente superior.

A partir del tamaño de cadenas 10, el coste temporal del algoritmo de recursividad clásica empieza a escalar rápidamente, pasando al orden de las décimas de segundo, los segundos y, finalmente, en 17, el orden de los minutos. La extrapolación de estos datos sugiere que, para tamaños a partir de 17, se vuelve inasumible. Por ello, no se analizarán valores superiores a este.

Cabe destacar que el uso de memoria se del algoritmo recursivo se mantiene notablemente bajo. Esto es así porque no realiza reservas de memoria como los otros dos algoritmos y su uso se restringe a poco más que la propia pila de ejecución.

En cuanto a los otros dos algoritmos, el tiempo de procesado se mantiene estable, incluso aumentando el número de caracteres de las secuencias. Siempre en el orden de los microsegundos y siempre, aproximadamente el doble de rápido en programación dinámica tabular que utilizando Hirschberg. En cuanto al uso de memoria, se mantiene también estable, incrementándose ligeramente con cada aumento del tamaño de cadenas, pero siempre ligeramente superior en programación dinámica tabular.

3.2.- Comportamiento para tamaños grandes (20 o más)

Ya se ha comentado que la recursividad clásica se omitirá a partir de este momento. Los dos algoritmos restantes.

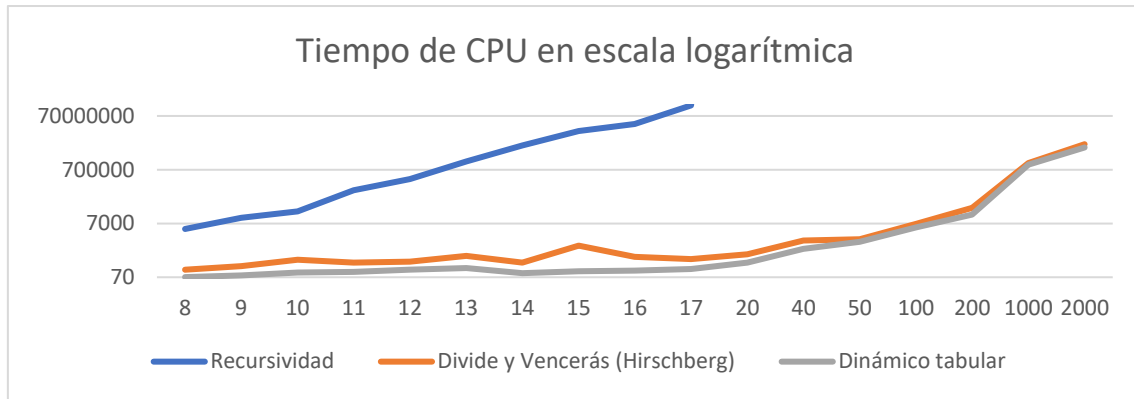
Si observamos el comportamiento cuando n se mantiene en el orden de las decenas (20, 40 y 50) el coste temporal crece en proporción al aumento del tamaño. Por ejemplo, la relación entre 20 y 40 podría parecer del doble pero en realidad, al tratarse de dos cadenas, al doblar el tamaño de las cadenas debería multiplicarse por cuatro el tiempo de procesado. Y eso es precisamente lo que pasa; Hirschberg pasa de unos 400 microsegundos a 1600 y la programación dinámica tabular de 200 a casi 800.

Es en el uso de memoria donde empezamos a ver diferencias, Hirschberg muestra un crecimiento muy moderado, apenas perceptible en este orden de tamaños, pero la programación dinámica tabular escala proporcionalmente al aumento del tamaño de n .

Cuando pasamos a los tamaños de n en el rango de las centenas y los millares, esta diferencia se va haciendo más evidente, necesitando reservar algunos MB para programación dinámica tabular mientras que Hirschberg solo necesita algo más de 100KB.

De este modo, se puede observar claramente que estos dos algoritmos presentan una complejidad temporal polinómica. En el plano teórico su complejidad sería $O(n \cdot m)$, aunque no siempre se aprecia este crecimiento de forma estricta debido a la sobrecarga del lenguaje y el entorno de ejecución.

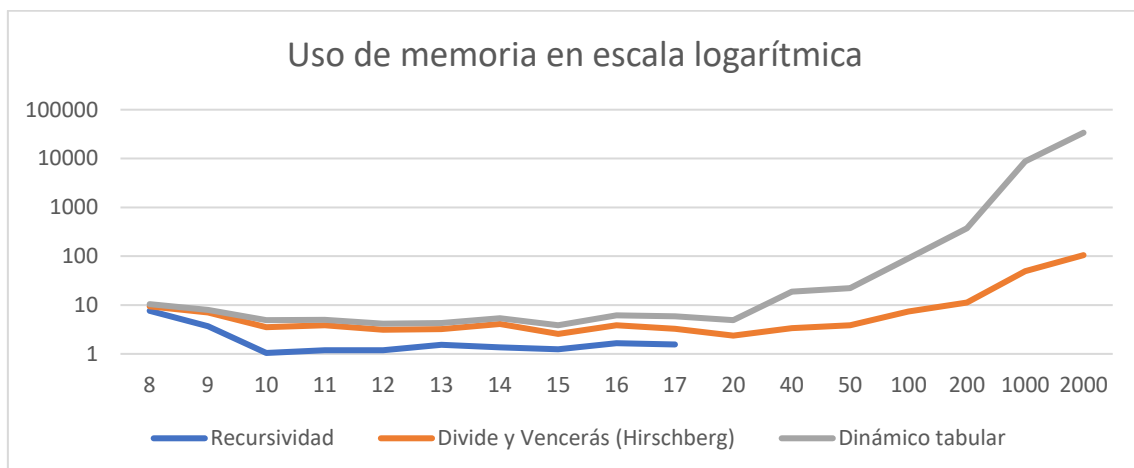
Estas tendencias pueden observarse con mayor claridad en la siguiente gráfica:



La versión tabular es, en todo momento, más rápida que Hirschberg. En conjuntos de datos grandes se observan mejoras de alrededor de un 25-30%. Pero esta mejora es solo posible a costa de un incremento masivo en el uso de memoria que, en esos mismo tamaños de secuencias, es de alrededor de 30 veces superior.

Esto es debido a que Hirschberg solo reserva dos vectores mientras que el método tabular reserva una matriz completa de tamaño $m \times n$. De este modo, queda claro que Hirschberg escala de forma lineal ($O(n + m)$) mientras que el método tabular lo hace de forma cuadrática ($O(n \cdot m)$)

En la siguiente figura, se muestra de forma gráfica este comportamiento:



4.- Discusión de los resultados

Los resultados obtenidos confirman las diferencias teóricas entre los tres algoritmos:

El algoritmo recursivo presenta un crecimiento muy rápido en el tiempo de ejecución a medida que se aumenta el tamaño de las cadenas. A partir de valores relativamente bajos se vuelve inasumible, llegando al orden de minutos para 17 caracteres. Sin embargo, su consumo de memoria se mantiene estable y bajo. Esto se debe a que su uso de la memoria se limita a la propia pila de ejecución.

Por su parte, el algoritmo de divide y vencerás de Hirschberg presenta un crecimiento temporal polinómico, dada su complejidad $O(n \cdot m)$. La aplicación de programación dinámica con divide y vencerás consigue reducir drásticamente el espacio de cálculo. Ya en tamaños pequeños se muestra más rápido que la recursividad aunque ligeramente más lento que la programación dinámica tabular. Para tamaños de secuencia grandes sigue siendo totalmente viable. Sin

embargo su punto fuerte es el uso de memoria. Este algoritmo muestra un uso muy contenido ya que sólo utiliza dos vectores. Esto hace que escale de forma aproximadamente lineal al aumentar los tamaños de secuencia.

Por último, la programación dinámica tabular es el algoritmo más rápido de forma consistente para cualquier tamaño de secuencia mostrándose siempre entre un 25% y un 30% más rápido que éste en tamaños de secuencia grandes. Sin embargo, esa mejora tiene una contrapartida importante en el uso de memoria. Al utilizar una matriz completa la memoria necesaria aumenta de manera cuadrática, haciéndolo poco adecuado para secuencias masivas.

En resumen, los resultados empíricos muestran el dilema clásico del diseño de algoritmos: lograr un compromiso entre uso de memoria y coste temporal. Se observa claramente que el algoritmo más rápido es el menos eficiente en el uso de la memoria y que el más eficiente en términos de memoria es, precisamente, el más lento.

5.- Conclusiones

Durante esta actividad se han comparado empíricamente tres enfoques diferentes para abordar el problema clásico de la Subsecuencia Común más Larga (LCS). Los resultados de las pruebas confirman las diferencias que, desde el punto de vista teórico, hay entre ellos.

El algoritmo recursivo, el más sencillo en la implementación, presenta una complejidad temporal exponencial en el peor caso, incluso con tamaños de secuencia relativamente pequeños. Como aspecto positivo, se debe destacar su mínimo consumo de memoria. Es totalmente inutilizable en casi cualquier caso real. Podría ser adecuado en entornos con tamaño de secuencia pequeños y memoria limitada, pero sin ninguna superioridad tangible respecto a los otros dos.

El algoritmo de Hirschberg presenta un equilibrio muy adecuado entre uso de memoria y tiempo de ejecución para secuencias de cualquier tamaño.

En cuanto al algoritmo de programación dinámica tabular, obtiene tiempos de ejecución sensiblemente mejores a los de Hirschberg pero su uso de memoria se dispara en tamaños de secuencia grandes, creciendo de manera cuadrática. Esto lo podría hacer útil en entornos donde el tiempo de respuesta sea vital, siempre que el tamaño de secuencia no sea masivo.

En conclusión, no existe un algoritmo que sea netamente superior a los demás en cualquier tipo de aplicación; unos son más rápidos y otros hacen mejor uso de la memoria. Trabajando con secuencias pequeñas, cualquiera de los tres algoritmos es válido. Con secuencias grandes, Hirschberg es claramente superior cuando la memoria es limitada. Y en los casos en que se pueda hacer uso “libre” de la memoria, la programación dinámica tabular sería la mejor opción. Por todo ello, se concluye que el algoritmo más flexible en términos globales es el de Hirschberg, ya que proporciona un rendimiento más que aceptable con cualquier tamaño de secuencia, manteniendo el uso de memoria en valores muy bajos.

6.- Anexos

6.1.- Anexo 1 – Tabla de resultados

Tamaño	Tiempo de CPU			Uso de memoria		
Algoritmo	Recursividad	Divide y Vencerás (Hirschberg)	Dinámico tabular	Recursividad	Divide y Vencerás (Hirschberg)	Dinámico tabular
8	4380,600061	133,7002032	70,89995779	7,622070313	9,431640625	10,42285156
9	11267,20011	179,5000862	82,6001633	3,7109375	7,080078125	7,98828125
10	19832,00014	317,5002057	104,7002152	1,044921875	3,53125	4,89453125
11	121616,9999	245,6000075	109,7999047	1,194335938	3,864257813	5,01953125
12	312064,2998	263,5999117	132,7001955	1,194335938	3,107421875	4,12890625
13	1430466,5	431,8999127	153,4998883	1,54296875	3,20703125	4,255859375
14	5665171,5	241,9999801	100,2999488	1,3515625	4,09765625	5,400390625
15	19489199,9	1051,79986	117,9999672	1,249023438	2,557617188	3,859375
16	35292865,5	403,2999277	124,5001331	1,653320313	3,840820313	6,15625
17	170430999,5	332,2998527	141,7999156	1,5625	3,280273438	5,853515625
20	1E+27	496,4999389	246,6998994	0	2,365234375	4,895507813
40	1E+27	1645,399956	797,0000152	0	3,375	18,67480469
50	1E+27	1831,099857	1455,599908	0	3,8359375	22,28417969
100	1E+27	6813,800195	5065,599922	0	7,401367188	90,01171875
200	1E+27	26411,20018	15089,99988	0	11,32421875	373,9082031
1000	1E+27	1226250,9	1069977,1	0	49,56152344	8742,948242
2000	1E+27	6229857,8	4715083,8	0	105,15625	33836,43359

6.2.- Enlace a repositorio

Tanto el código fuente como los archivos de dataset y resultados de la ejecución en el equipo del autor están disponibles en su repositorio de GitHub en el siguiente enlace:

<https://github.com/erobmar/AnalizadorLCS>